Software Engineering Lab 5: Static Code Analysis

Finding and fixing issues in a Python script with Pylint, Bandit and Flake8

Name: Abhigyan Dutta
SRN: PES2UG24CS019
Class: 5A, CSE
Date: 28 October 2026
Assigned program: L5_17, Apartment Society Maintenance Dues Ledger (society_dues.py)
Repo: github.com/UltraBot05/se-ps17-vaccination-scheduling, folder lab5-static-python

1. The program

society_dues.py is a small dues ledger for a housing society. It keeps an in-memory SQLite table of dues, bills flats for a month, marks a due as paid, works out a late fee from a rule string and lists the flats that still owe money. Running it prints 2500, 100 and ['A-102'].

It works, but the code had SQL built with f-strings, an eval on the fee rule, a password in the source, and a few quality and style problems.

2. Running the tools

I set up a venv on Windows (Python 3.14.6) and installed pylint, bandit and flake8. The first reports were saved as *_before.txt from the original file. After each round of fixes the tool was run again into *_after.txt.

Tool	Before	After
Pylint	5.81/10, 13 messages	10.00/10
Bandit	3 issues (2 Medium, 1 Low)	No issues identified
Flake8	5 issues	0 issues

3. Issues found and how I fixed them

Line numbers are from the original file. The full version of this table is in `issues.md` in the repo.

#	Issue	Type	Tool and code	Line(s)	Fix
1	SQL injection	Security, Medium	Bandit B608	9	INSERT built with an f-string. Changed to ? placeholders with the values passed as a tuple.
2	eval on the fee rule	Security, Medium	Bandit B307, Pylint W0123	21	Any Python in the rule would run. Replaced with a small ast based calc() that only allows numbers and + - * /.
3	Hardcoded password	Security, Low	Bandit B105	5	Moved to the TREASURER_PASSWORD environment variable.
4	Mutable default history=[]	Bug	Pylint W0102	8	One list was shared by every call. Default is now None and a new list is made inside.
5	except Exception	Bug	Pylint W0718	22	Now catches only SyntaxError, ValueError and ZeroDivisionError.
6	Inconsistent return in collect	Bug	Pylint R1710	12-16	Returns None explicitly. Also added paid = 0 to the SELECT (see 10).
7	rows != None	Bug / Style	Pylint C0121, Flake8 E711	29	fetchall() never returns None, so the check was always true. Removed it.

#	Issue	Type	Tool and code	Line(s)	Fix
8	addDue not snake_case, no docstrings	Convention	Pylint C0103, C0114, C0116	1, 8 and each def	Renamed to add_due. Added a module docstring and one per function.
9	Long lines, missing blank lines	Style	Flake8 E501, E302, Pylint C0301	4, 12, 13, 15	Split the long execute calls inside brackets. Two blank lines before collect.
10	Same due collected twice	Bug	None of the tools	12-16	collect returned 2500 every time it was called. Now only unpaid rows are selected, so the second call gives None.

Row 10 is not from any report. I found it while fixing the inconsistent return in `collect`. It is probably the most serious bug in the file for an actual society, since it shows a flat paying twice.

The two security fixes

Before, line 9 and line 21:

```
conn.execute(f"INSERT INTO dues VALUES ('{flat}', '{month}', {amount}, 0)")
late_fee = eval(rule)
```

After:

```
conn.execute("INSERT INTO dues VALUES (?, ?, ?, 0)", (flat, month, amount))
late_fee = calc(ast.parse(rule, mode="eval").body)
```

`calc` walks the parsed rule and only accepts numbers and + - * /. Anything else raises ValueError and the fee is 0. Bandit suggested `ast.literal_eval`, but it rejects "50 * 2", so it would have broken the late fee feature.

4. Commits

One commit for each tool, as asked in the handout:

- **Lab 5: fix Bandit issues** (SQL injection, eval, hardcoded password), with all three before reports
- **Lab 5: fix Pylint issues** (mutable default, broad except, naming, docstrings)
- **Lab 5: fix Flake8 issues**, with the final after reports, issue table, reflection and this document

Fixing rows 2 and 7 cleared messages from two tools at once, so some fixes show up in an earlier commit than the tool that also reported them.

5. Reflection

Which issues were the easiest to fix, and which were the hardest? Why?

The easiest were the Flake8 style issues and the Pylint naming and docstring messages. The tool gives the exact line, and there is only one sensible fix. Renaming addDue only touched two calls in main.

The hardest was eval. Bandit's hint was ast.literal_eval, but that only reads plain values and cannot do 50 * 2, so following the hint as written would have removed the feature along with the security hole. I wrote a small calc function on top of ast.parse instead, and then had to work out which exceptions a bad rule can actually raise (1 + gives SyntaxError, 10/0 gives ZeroDivisionError, anything else gives ValueError) to replace except Exception.

Did the static analysis tools report any false positives? If so, describe one example.

B105 (hardcoded password) is close to one. Bandit flagged it only because the variable name has PASSWORD in it, and nothing in the program ever reads that variable. I still moved it to an environment variable, because a password in source code stays in git history once it is pushed.

Line length was also reported differently. Flake8 stops at 79 characters and Pylint at 100, so lines 4 and 15 were wrong for one tool and fine for the other. I followed the stricter limit.

The bigger point was what the tools missed. None of them found that collect could take the same payment twice. And rows != None was flagged as a style problem by two tools, when the real issue was that the check could never be false.

How would you integrate static analysis tools into your actual software development workflow?

Locally, a pre-commit hook running Flake8 and Bandit on changed files, since both are fast, and blocking any commit with a Medium or High Bandit issue. On the repo, a GitHub Actions job on every pull request running all three, with pylint --fail-under=9.5 and Bandit failing on Medium and above. A shared config file would also be needed so everyone uses the same line length.

This lab also showed that running the tools again after each fix matters. My new calc function was only checked by Pylint in the next round. If I had stopped after Bandit, nothing would have checked it.

What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

Pylint went from 5.81 to 10.00, and Bandit and Flake8 both ended with no issues. The output is still 2500, 100 and ['A-102']. I also checked a few things by hand. A flat name with a quote in it is now stored as text instead of being run as SQL. A fee rule like __import__("os") gives 0 instead of running code. A second collect on the same due returns None. Two add_due calls no longer share one history list.

What it did not fix: the connection is still a global and there are no tests. A 10.00 from Pylint means the code follows the rules Pylint knows. It does not mean the code is correct, and the double collection bug shows that.